

Testing Topological Memory

Storage capacity, retrieval accuracy, I/O performance

This test measures the memory retrieval accuracy as a function of the memory density.

Process: In each iteration, a new batch of random associations $A \rightarrow B$ is stored in memory.

We then retrieve the same batch with inputs A and compare the outputs with B, measuring overlaps and Hamming distances. Optionally, random noise may be added to inputs A, testing the retrieval accuracy in the presence of additive or subtractive noise.

We use this setup to verify that different memory backends (C, Mathematica, Python, ...) behave exactly the same.

Requirements

```
<< "Memory.m"
```

Configuration

```
testconfig = <|  
  "A_parameters" → {1000, 10},  
  "B_parameters" → {1001, 10},  
  "batchsize" → 50000,  
  "iterations" → 40,  
  "noise" → 0  
|>;
```

Test setup

```
RandomSet[{n_Integer, p_Integer}] := Sort[RandomSample[Range[n], p]]
```

```
RandomAssociation[config_Association] :=
  {RandomSet[config["A_parameters"]], RandomSet[config["B_parameters"]]}
```

```
AddNoise[ A : {___Integer}, {n_Integer, p_Integer}, noise_Integer] :=
  Which[
    noise == 0, A,
    noise > 0, Union[A, RandomInteger[{1, n}, noise]],
    noise < 0, Sort[RandomSample[A, Length[A] + noise]]
  ]
```

```
Overlap[ A : {___Integer}, B : {___Integer} ] :=
  Length[Intersection[A, B]]
```

```
Distance[ A : {___Integer}, B : {___Integer} ] :=
  Length[Complement[A, B]] + Length[Complement[B, A]]
```

```
Test[config_Association] :=
  Module[ {mem, size, data, inp, out, storetime, retrievetime,
    distance, overlap, memsize, cols},

    mem = Memory[config];

    memsize = mem["B_parameters"][[1]] *
      mem["A_parameters"][[1]] (mem["A_parameters"][[1]] - 1) / 2;

    size = config["batchsize"];

    Print[ "Configuration: ", config];
    Print[ "Memory: ", mem];
    Print[];

    cols = {"items", "writes/s", "density", "reads/s", "distance", "overlap",
      "storetime", "retrievetime", "distance", "overlap", "storetime", "retrievetime"};

    Print["|" <> StringJoin[StringPadLeft[# <> " ", 12] <> "|" & /@ cols];
    Print["|" <> StringRepeat["-----|", Length[cols]]];

    Do[

      data = Table[ RandomAssociation[mem], size];

      storetime = First[AbsoluteTiming[ mem["store"] @@@ data]];
```

```

inp = AddNoise[# , mem["A_parameters"], config["noise"]] & /@ data[[All
retrievetime = First[AbsoluteTiming[ out = mem["retrieve"] /@ inp ]];

distance = Total[Table[ Distance[out[[i]], data[[i,2]], {i, 1, size}]];

overlap = Total[Table[ Overlap[out[[i]], data[[i,2]], {i, 1, size}]];

Print[ "|",
      PaddedForm[ iter*size, 10], " |",
      PaddedForm[ Round[size/storetime],10], " |",
      StringPadLeft[ToString[DecimalForm[ N[mem["memorycount"]]/memsize,
PaddedForm[ Round[ size/retrievetime], 10], " |",
      PaddedForm[ N[distance/size], {9,6}], " |",
      PaddedForm[ N[overlap/size], {9,6}] , " |"
      ],

      {iter, 1, config["iterations"]}
]
]

```

Run test

```
Test[testconfig];
```

Configuration: <|A_parameters → {1000, 10},
 B_parameters → {1001, 10}, batchsize → 50 000, iterations → 40, noise → 0|>
 Memory: <|A_parameters → {1000, 10}, B_parameters → {1001, 10}, T → 7,
 store → store\$2007018, retrieve → retrieve\$2007018, clear → clear\$2007018,
 memorycount → memorycount\$2007018, backend → Mathematica|>

items	writes/s	density	reads/s	distance	overlap
50000	12169	0.044004	9327	0.000000	10.000000
100000	11252	0.086073	9017	0.000000	10.000000
150000	11229	0.126288	8851	0.000000	10.000000
200000	11212	0.164731	8663	0.000000	10.000000
250000	11077	0.201481	8801	0.000000	10.000000
300000	11098	0.236622	8942	0.000000	10.000000
350000	11130	0.270217	9017	0.000000	10.000000

	400000		11122		0.302326		9011		0.000000		10.000000	
	450000		11096		0.333026		9016		0.000000		10.000000	
	500000		11102		0.362370		8929		0.000000		10.000000	
	550000		11098		0.390426		8982		0.000000		10.000000	
	600000		11133		0.417253		8866		0.000000		10.000000	
	650000		11053		0.442895		8899		0.000000		10.000000	
	700000		11045		0.467417		8892		0.000000		10.000000	
	750000		11072		0.490851		8925		0.000000		10.000000	
	800000		11036		0.513247		8978		0.000000		10.000000	
	850000		11068		0.534667		9006		0.000000		10.000000	
	900000		10988		0.555143		8941		0.000000		10.000000	
	950000		10993		0.574721		8975		0.000000		10.000000	
	1000000		10954		0.593435		8988		0.000000		10.000000	
	1050000		11088		0.611329		8898		0.000000		10.000000	
	1100000		10799		0.628431		8765		0.000000		10.000000	
	1150000		10963		0.644776		8623		0.000000		10.000000	
	1200000		10974		0.660409		8528		0.000020		10.000000	
	1250000		10754		0.675358		8866		0.000040		10.000000	
	1300000		10525		0.689641		8908		0.000060		10.000000	
	1350000		10681		0.703298		8900		0.000220		10.000000	
	1400000		10639		0.716345		8699		0.000520		10.000000	
	1450000		10778		0.728830		8836		0.000940		10.000000	
	1500000		10931		0.740761		8952		0.002280		10.000000	
	1550000		10817		0.752166		8966		0.004000		10.000000	
	1600000		10774		0.763071		8941		0.008200		10.000000	
	1650000		10997		0.773496		8872		0.013460		10.000000	
	1700000		10888		0.783464		8855		0.024520		10.000000	
	1750000		11105		0.792995		8847		0.039500		10.000000	
	1800000		10939		0.802098		8875		0.067940		10.000000	
	1850000		11006		0.810806		8852		0.105820		10.000000	
	1900000		10897		0.819137		8873		0.160400		10.000000	
	1950000		10945		0.827097		8843		0.243580		10.000000	
	2000000		10964		0.834703		8873		0.364720		10.000000	